



Dockerizing Spring Boot App with PostgreSQL using Docker Compose

Multi-Container Application Setup – Creative Theory Notes

1. From Single Container to Multi-Container Architecture

Earlier examples involved running:

- One application
- Inside one container

In real-world applications:

- Applications depend on databases
- Databases must run independently
- Each service runs in its **own container**

In this project:

- **Spring Boot Student App** → App container
- **PostgreSQL** → Database container

This separation improves scalability, reliability, and maintainability.

2. Initial Data Loading (Development Context)

The application uses a **SQL file** to preload student data.

Key note:

- This is **only for testing/demo**
- In real applications:
 - Databases start empty
 - Users register data via UI or APIs

This distinction is important when moving from development to production.

3. Reusing and Updating Dockerfile

The speaker reuses an existing **Dockerfile** and adapts it for the Student App.

Important adjustments:

- Update project name

- Ensure correct JAR file name
- Avoid long default artifact names

Reason:

- Maven generates long JAR names by default
 - Explicit naming makes Dockerfile cleaner and less error-prone
-

4. Building the JAR File

Before Dockerizing, the application must be packaged.

Command used:

- **mvn clean package**

Purpose:

- Cleans previous builds
- Generates a fresh executable JAR
- Places JAR in the **target** folder

This JAR is later copied into the Docker image.

5. Why Docker Compose Is Needed

Running multiple containers manually requires:

- Multiple **docker run** commands
- Manual network setup
- Environment variable configuration

Docker Compose simplifies this by:

- Managing multiple containers together
 - Using one configuration file
 - Running everything with a single command
-

6. Introduction to **docker-compose.yml**

Docker Compose uses a file named:

- **docker-compose.yml**

File format:

- **YAML (Yet Another Markup Language)**

Advantages of YAML:

- Clean and readable
- Supports nested configuration
- Easier than long `.properties` files

This file defines:

- Containers (services)
 - Networks
 - Ports
 - Environment variables
-

7. Docker Compose Versioning

The speaker recommends:

- **version: 3.7 or 3.8**

Reason:

- Stable
 - Widely supported
 - Suitable for most Docker setups
-

8. Defining Services in Docker Compose

Docker Compose works around the concept of **services**.

In this setup:

- **app** → Spring Boot application
- **postgres** → PostgreSQL database

Each service runs in its own container but shares a Docker network.

9. Configuring the App Service

Key configurations for the application container:

- Built using a **Dockerfile**
- Exposes application port
- Host port **8090** → Container port **8080**

This avoids conflicts with other local applications.

10. Configuring PostgreSQL Service

The PostgreSQL container is configured using:

- Official **Postgres image**
- Latest stable version

Important settings include:

- Database name
- Username
- Password

These are passed as **environment variables**, which differ from local database settings.

11. Avoiding Port Conflicts for PostgreSQL

To prevent conflicts with local PostgreSQL:

- Container port: **5432** (default)
- Host port: **5433**

This allows:

- Local Postgres (if any)
- Docker Postgres

To run simultaneously without issues.

12. Running Multi-Container App with Docker Compose

Docker Compose is used to:

- Build images
- Create containers
- Start services

Initial issues encountered:

- Containers already running
- Forgetting to stop previous processes

Solution:

- Stop existing containers
 - Clean and rebuild
-

13. Debugging Connection Errors

After startup:

- Containers run without crashing
- Application fails to connect to database

This introduces an important learning moment:

👉 **Debugging is a core Docker skill**

14. Understanding Docker Networking

Docker containers:

- Do NOT share `localhost`
- Communicate using **container names**
- Must be on the **same Docker network**

Problem identified:

- App configuration still points to `localhost`
 - PostgreSQL is running in a separate container
-

15. Fixing Database Connection Configuration

To fix connectivity:

- Update DB URL to use **Postgres service name**
- Update username and password
- Match credentials defined in Docker Compose

Example concept:

- ❌ `localhost`
- ✅ `postgres` (service name)

This allows container-to-container communication.

16. Handling Maven Build Issues

Build errors occur due to:

- Test failures
- Environment mismatch

Temporary workaround:

- Skip tests during build using:
 - `-DskipTests`

Important note:

- Acceptable for learning/demo
 - **Not recommended for production**
-

17. Cleaning and Rebuilding Containers

To ensure a clean run:

- Stop running containers
- Remove old containers
- Rebuild images
- Restart using Docker Compose

This avoids stale configuration issues.

18. Verifying Container Status

To troubleshoot:

- `docker ps` is used
- Confirms which containers are running
- Helps identify startup or crash issues

This confirms:

- App container is running
 - Postgres container is running
-

19. Key Learning Outcome So Far

At this stage, the project has:

- Spring Boot app containerized
- PostgreSQL containerized
- Docker Compose YAML created
- Multi-container architecture defined

Remaining challenge:

- Ensuring correct networking & configuration
-

20. Conclusion

This section demonstrates how real-world backend systems are deployed using Docker Compose. While initial errors occur, they highlight critical concepts such as:

- Container networking
- Service discovery via names
- Environment-based configuration

Docker Compose enables developers to manage complex, multi-service applications efficiently using a single configuration file.

If you want next, I can:

- Write the **final working docker-compose.yml**
- Fix **Spring Boot DB config for Docker**
- Explain **Docker networking visually**
- Show **best practices for Compose**
- Turn this into a **mini project report or lab manual** 📄📦